

For Hive Lab Use Case, write in the proper format if we need new terminal tell that also, whichever we are change from system to system tell that me "<<>>" write that inside it so we know that when what we have to change as per changes in use cases or system get change.

Here is the complete step-by-step procedure for **Hive Lab Use Case 4: Online Learning Platform Analytics** organized by Group/Terminal context. All system-dependent values (such as paths, usernames, database names, table names, file names, hostnames, and ports) that need to be updated when systems or use cases change are enclosed in << >>.

TXT +1

Group A: System & Storage Operations (Linux / HDFS)

Terminal 1: Hadoop & Local File Preparation

1. Start Hadoop Services & Verify Processes:

Bash



```
start-dfs.sh
start-yarn.sh
jps
hdfs dfs -ls /
```

1. Create Local Working Directories:

Bash



```
mkdir -p ~/<<hive-online-learning>>/<<data>>[cite: 2]
cd ~/<<hive-online-learning>>/<<data>>[cite: 2]
```

1. Create CSV Data Files: Create learners.csv:

[cite: 2]

Bash



```
nano <<learners.csv>>[cite: 2]
```

File Content:

Plaintext



```
<<learner_id>>,<<learner_name>>[cite: 2]  
<<L001>>,<<Rahul>>[cite: 2]  
<<L002>>,<<Priya>>[cite: 2]  
<<L003>>,<<Arun>>[cite: 2]  
<<L004>>,<<Neha>>[cite: 2]  
<<L005>>,<<Kiran>>[cite: 2]
```

Create `courses.csv`:

[cite: 2]

Bash



```
nano <<courses.csv>>[cite: 2]
```

File Content:

Plaintext



```
<<course_id>>,<<course_name>>,<<category>>[cite: 2]  
<<C001>>,<<Python>>,<<Programming>>[cite: 2]  
<<C002>>,<<Hadoop>>,<<BigData>>[cite: 2]
```

```
<<C003>>,<<Hive>>,<<BigData>>[cite: 2]  
<<C004>>,<<Statistics>>,<<DataScience>>[cite: 2]
```

Create `enrollments.csv`:

[cite: 2]

Bash



```
nano <<enrollments.csv>>[cite: 2]
```

File Content:

Plaintext



```
<<enrollment_id>>,<<learner_id>>,<<course_id>>,<<enrollment_date>>[cite: 2]  
<<E001>>,<<L001>>,<<C001>>,<<2026-10-01>>[cite: 2]  
<<E002>>,<<L001>>,<<C002>>,<<2026-10-02>>[cite: 2]  
<<E003>>,<<L002>>,<<C001>>,<<2026-10-02>>[cite: 2]  
<<E004>>,<<L003>>,<<C003>>,<<2026-10-03>>[cite: 2]  
<<E005>>,<<L004>>,<<C004>>,<<2026-10-03>>[cite: 2]
```

1. Verify Local CSV Files:

Bash



```
cat <<learners.csv>>[cite: 2]  
cat <<courses.csv>>[cite: 2]  
cat <<enrollments.csv>>[cite: 2]
```

1. Create HDFS Target Directories:

Bash



```
hdfs dfs -mkdir -p /<<online-learning>>/raw/<<learners>>[cite: 2]
hdfs dfs -mkdir -p /<<online-learning>>/raw/<<courses>>[cite: 2]
hdfs dfs -mkdir -p /<<online-learning>>/raw/<<enrollments>>[cite: 2]
```

1. Upload Data from Local Filesystem to HDFS:

Bash



```
hdfs dfs -put -f <<learners.csv>> /<<online-learning>>/raw/<<learners>>/[cite: 2]
hdfs dfs -put -f <<courses.csv>> /<<online-learning>>/raw/<<courses>>/[cite: 2]
hdfs dfs -put -f <<enrollments.csv>> /<<online-learning>>/raw/<<enrollments>>/[cite: 2]
```

1. Verify Uploaded Files in HDFS:

Bash



```
hdfs dfs -ls /<<online-learning>>/raw/<<learners>>[cite: 2]
hdfs dfs -ls /<<online-learning>>/raw/<<courses>>[cite: 2]
hdfs dfs -ls /<<online-learning>>/raw/<<enrollments>>[cite: 2]
```

Group B: Hive & Data Warehouse Analytics

Terminal 2: Open New Terminal for Hive / Beeline CLI Shell

1. Launch Hive Client / Service Connection:

Bash



```
beeline -u jdbc:hive2://<<localhost>>:<<10000>>/<<default>>[cite: 2]
```

(Alternative interactive shell: `hive`)

[cite: 2]

1. Create Database and Switch Context:

SQL



```
CREATE DATABASE IF NOT EXISTS <<online_learning>>;[cite: 2]
USE <<online_learning>>;[cite: 2]
```

1. Create Managed (Internal) Table & Load Data via Local Path:

SQL



```
CREATE TABLE <<learners>> (
    <<learner_id>> STRING,
    <<learner_name>> STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '<<, >>';[cite: 2]

LOAD DATA LOCAL INPATH '/home/<<ubuntu>>/<<hive-online-learning>>/data/<<learners.csv>>'
INTO TABLE <<learners>>;[cite: 2]

SELECT * FROM <<learners>>;[cite: 2]
```

1. Create External Tables pointing directly to HDFS Locations:

SQL



```
CREATE EXTERNAL TABLE <<courses>> (
    <<course_id>> STRING,
    <<course_name>> STRING,
    <<category>> STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '<<, >>'
LOCATION '/<<online-learning>>/raw/<<courses>>';[cite: 2]

CREATE EXTERNAL TABLE <<enrollments>> (
    <<enrollment_id>> STRING,
    <<learner_id>> STRING,
    <<course_id>> STRING,
    <<enrollment_date>> DATE
)
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY '<<, >>'  
LOCATION '/<<online-learning>>/raw/<<enrollments>>';[cite: 2]  
  
SHOW TABLES;[cite: 2]  
SELECT * FROM <<courses>>;[cite: 2]  
SELECT * FROM <<enrollments>>;[cite: 2]
```

1. Perform Join Operations:

Inner Join:

SQL



```
SELECT l.<<learner_id>>, l.<<learner_name>>, c.<<course_name>>  
FROM <<learners>> l  
INNER JOIN <<enrollments>> e ON l.<<learner_id>> = e.<<learner_id>>  
INNER JOIN <<courses>> c ON e.<<course_id>> = c.<<course_id>>;[cite: 2]
```

Left Outer Join (All learners with/without enrollments):

SQL



```
SELECT l.<<learner_id>>, l.<<learner_name>>, e.<<course_id>>  
FROM <<learners>> l  
LEFT OUTER JOIN <<enrollments>> e ON l.<<learner_id>> = e.<<learner_id>>;[cite: 2]
```

Right Outer Join:

SQL



```
SELECT l.<<learner_id>>, l.<<learner_name>>, e.<<course_id>>  
FROM <<learners>> l  
RIGHT OUTER JOIN <<enrollments>> e ON l.<<learner_id>> = e.<<learner_id>>;[cite: 2]
```

Full Outer Join:

SQL



```
SELECT l.<<learner_id>>, l.<<learner_name>>, e.<<course_id>>
FROM <<learners>> l
FULL OUTER JOIN <<enrollments>> e ON l.<<learner_id>> = e.<<learner_id>>;[cite: 2]
```

1. Analytics & Problem-Solving Queries:

Find Learners with NO Enrollment:

SQL



```
SELECT l.<<learner_id>>, l.<<learner_name>>
FROM <<learners>> l
LEFT JOIN <<enrollments>> e ON l.<<learner_id>> = e.<<learner_id>>
WHERE e.<<learner_id>> IS NULL;[cite: 2]
```

Find Courses with NO Learners:

SQL



```
SELECT c.<<course_id>>, c.<<course_name>>
FROM <<courses>> c
LEFT JOIN <<enrollments>> e ON c.<<course_id>> = e.<<course_id>>
WHERE e.<<course_id>> IS NULL;[cite: 2]
```

Find Courses with Highest Enrollments:

SQL



```
SELECT c.<<course_id>>, c.<<course_name>>, COUNT(e.<<enrollment_id>>) AS <<enrollment_count>>
FROM <<courses>> c
JOIN <<enrollments>> e ON c.<<course_id>> = e.<<course_id>>
GROUP BY c.<<course_id>>, c.<<course_name>>
ORDER BY <<enrollment_count>> DESC;[cite: 2]
```

1. Join Optimization Settings:

Map-Side Join (Auto convert small tables):

SQL



```
SET hive.auto.convert.join=true;[cite: 2]
SELECT e.<<enrollment_id>>, e.<<learner_id>>, c.<<course_name>>
FROM <<enrollments>> e
JOIN <<courses>> c ON e.<<course_id>> = c.<<course_id>>;[cite: 2]
```

Reduce-Side Join (Force standard MapReduce reduce-side join):

SQL



```
SET hive.auto.convert.join=false;[cite: 2]
SELECT e.<<enrollment_id>>, e.<<learner_id>>, c.<<course_name>>
FROM <<enrollments>> e
JOIN <<courses>> c ON e.<<course_id>> = c.<<course_id>>;[cite: 2]
```

1. Partitioned Table Operations:

Create Partitioned Table:

SQL



```
CREATE TABLE <<enrollments_partitioned>> (
    <<enrollment_id>> STRING,
    <<learner_id>> STRING,
    <<course_id>> STRING
)
PARTITIONED BY ( <<enrollment_date>> DATE )
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '<<, >>';[cite: 2]
```

Insert Data Into Dynamic/Static Partitions:

SQL



```
INSERT INTO TABLE <<enrollments_partitioned>> PARTITION (<<enrollment_date>>='<<2026-10-02>>')
SELECT <<enrollment_id>>, <<learner_id>>, <<course_id>> FROM <<enrollments>> WHERE <<enrollment_date>>='<<2026-10-02>>';

INSERT INTO TABLE <<enrollments_partitioned>> PARTITION (<<enrollment_date>>='<<2026-10-02>>')
SELECT <<enrollment_id>>, <<learner_id>>, <<course_id>> FROM <<enrollments>> WHERE <<enrollment_date>>='<<2026-10-02>>';

INSERT INTO TABLE <<enrollments_partitioned>> PARTITION (<<enrollment_date>>='<<2026-10-02>>')
SELECT <<enrollment_id>>, <<learner_id>>, <<course_id>> FROM <<enrollments>> WHERE <<enrollment_date>>='<<2026-10-02>>';
```

Show & Query Partitions:

SQL



```
SHOW PARTITIONS <<enrollments_partitioned>>;[cite: 2]

SELECT * FROM <<enrollments_partitioned>> WHERE <<enrollment_date>>='<<2026-10-02>>';
```

Combine Partition Pruning with Joins:

SQL



```
SELECT e.<<enrollment_id>>, e.<<learner_id>>, c.<<course_name>>
FROM <<enrollments_partitioned>> e
JOIN <<courses>> c ON e.<<course_id>> = c.<<course_id>>
WHERE e.<<enrollment_date>>='<<2026-10-02>>';[cite: 2]
```

explain each join diagrammatically which we are doing here with that operation we did

Here is a step-by-step diagrammatic explanation of each join operation performed in the Hive Lab Use Case, mapped directly to the actual dataset and queries.

Input Data Overview

learners Table:

learner_id	learner_name
L001	Rahul
L002	Priya
L003	Arun
L004	Neha
L005	Kiran
...	

courses Table:

course_id	course_name	category
C001	Python	Programming
C002	Hadoop	BigData
C003	Hive	BigData
C004	Statistics	DataScience
...		

enrollments Table:

enrollment_id	learner_id	course_id	enrollment_date
E001	L001	C001	2026-10-01
E002	L001	C002	2026-10-02
E003	L002	C001	2026-10-02
E004	L003	C003	2026-10-03
E005	L004	C004	2026-10-03
...			

1. Inner Join

- **Operation:** Returns only the matching records present in all joined tables.
- **Query Executed:**

SQL



```
SELECT l.learner_id, l.learner_name, c.course_name
FROM learners l
INNER JOIN enrollments e ON l.learner_id = e.learner_id
INNER JOIN courses c ON e.course_id = c.course_id;
```

- **Venn Diagram Representation:**

Plaintext



Table A (learners / enrollments)			Table B (courses)		
+-----+			+-----+		

	Unmatched		* *		Unmatched
	Learners		* MATCH*		Courses
	(L005)		* *		(None)

+-----+		+-----+
[MATCHED]		

- Query Result:

learner_id	learner_name	course_name
:--	:--	:--
L001	Rahul	Python
L001	Rahul	Hadoop
L002	Priya	Python
L003	Arun	Hive
L004	Neha	Statistics

2. Left Outer Join

- Operation:** Returns ALL records from the Left table (**learners**), along with matching values from the Right table (**enrollments**). If no match exists on the right, **NULL** is populated.

TXT

- Query Executed:

SQL



```
SELECT l.learner_id, l.learner_name, e.course_id
FROM learners l
LEFT OUTER JOIN enrollments e ON l.learner_id = e.learner_id;
```

- Venn Diagram Representation:

Plaintext



Table A (learners)		Table B (enrollments)	
+-----+		+-----+	
ALL LEFT RECORDS INCLUDED	*****	Unmatched	
(L001, L002, L003, L004)	MATCH*	Enrollments	
+ L005 (Gets NULL)	*****	(None)	

+-----+	+-----+	+-----+	+-----+

- Query Result:

learner_id learner_name course_id
:--- :--- :---
L001 Rahul C001
L001 Rahul C002
L002 Priya C001
L003 Arun C003
L004 Neha C004
L005 Kiran NULL

3. Right Outer Join

- Operation: Returns ALL records from the Right table (**enrollments**), along with matching values from the Left table (**learners**).

TXT

- Query Executed:

SQL



```
SELECT l.learner_id, l.learner_name, e.course_id
FROM learners l
RIGHT OUTER JOIN enrollments e ON l.learner_id = e.learner_id;
```

- Venn Diagram Representation:

Plaintext



Table A (learners)		Table B (enrollments)	
+-----+		+-----+	
Unmatched	****	ALL RIGHT RECORDS INCLUDED	
Learners	*MATCH	(E001, E002, E003,	
(L005)	****	E004, E005)	
+-----+		+-----+	

- **Query Result:**
| learner_id | learner_name | course_id |
| :--- | :--- | :--- |
| L001 | Rahul | C001 |
| L001 | Rahul | C002 |
| L002 | Priya | C001 |
| L003 | Arun | C003 |
| L004 | Neha | C004 |

4. Full Outer Join

- **Operation:** Returns **ALL** records from **BOTH** tables (`learners` and `enrollments`). Unmatched rows on either side are filled with `NULL`.
TXT
- **Query Executed:**

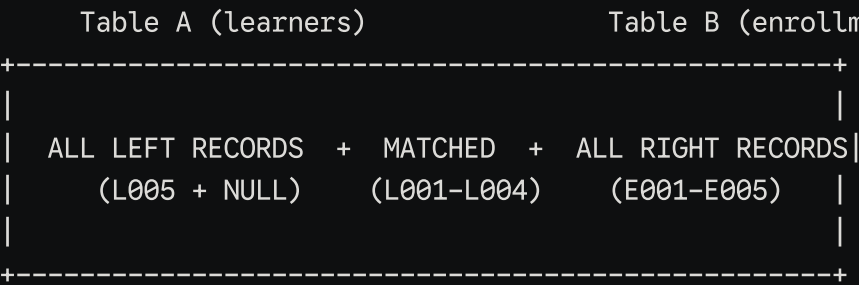
SQL



```
SELECT l.learner_id, l.learner_name, e.course_id
FROM learners l
FULL OUTER JOIN enrollments e ON l.learner_id = e.learner_id;
```

- **Venn Diagram Representation:**

Plaintext



- **Query Result:**
| learner_id | learner_name | course_id |
| :--- | :--- | :--- |
| L001 | Rahul | C001 |
| L001 | Rahul | C002 |

```
| L002 | Priya | C001 |
| L003 | Arun | C003 |
| L004 | Neha | C004 |
| L005 | Kiran | NULL |
```

5. Anti-Join Operations (Finding Unmatched Records)

A. Learners with NO Enrollments

- Query Executed:

SQL



```
SELECT l.learner_id, l.learner_name
FROM learners l
LEFT JOIN enrollments e ON l.learner_id = e.learner_id
WHERE e.learner_id IS NULL;
```

- Diagram:

Plaintext



Table A (learners)		Table B (enrollments)
+-----+		+-----+
*** ONLY ***		
UNMATCHED	(MATCH)	Enrollments
LEARNERS		
(L005)		
+-----+		+-----+

- Query Result: L005 | Kiran

B. Courses with NO Learners

- Query Executed:

SQL



```
SELECT c.course_id, c.course_name
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
WHERE e.course_id IS NULL;
```

- Diagram:

Plaintext



Table A (courses)		Table B (enrollments)
+-----+		+-----+
*** ONLY ***		
UNMATCHED	(MATCH)	Enrollments
COURSES		
(None)		
+-----+		+-----+

- Query Result: 0 rows returned (all courses C001 – C004 have active enrollments).

6. Execution Engine Join Strategies

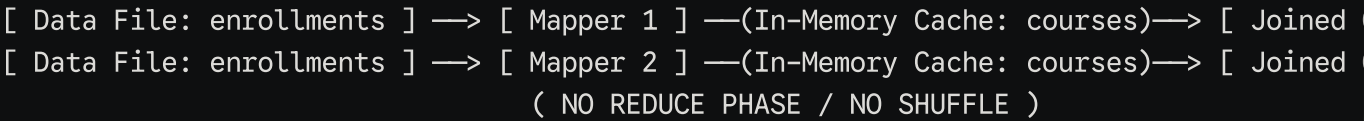
A. Map-Side Join (hive.auto.convert.join=true)

- Concept: When small reference tables (e.g., courses or learners) can fit into memory, Hive caches them across all Mappers. The join is processed entirely during the Map Phase without a Reduce Phase.

TXT

- Execution Diagram:

Plaintext



B. Reduce-Side Join (hive.auto.convert.join=false)

- **Concept:** Used when both tables are large. Data from both tables sharing the same Join Key (e.g., `course_id`) is shuffled across the network and combined at the **Reduce Phase**.

TXT

- **Execution Diagram:**

Plaintext



```
[ Table: enrollments ] → [ Mapper 1 ] ┘  
                                     |→ [ Network Shuffle ] → [ Reducer ] →  
[ Table: courses ]      → [ Mapper 2 ] ┘
```